

IIO 子系统

一、IIO 子系统：

工业场合里面也有大量的模拟量和数字量之间的转换，也就是我们常说的 ADC 和 DAC。而且随着手机、物联网、工业物联网和可穿戴设备的爆发，传感器的需求只持续增强。比如手机或者手环里面的加速度计、光传感器、陀螺仪、气压计、磁力计等，这些传感器本质上都是 ADC，查看这些传感器的手册，会发现他们内部都会有个 ADC，传感器对外提供 IIC 或者 SPI 接口，SOC 可以通过 IIC 或者 SPI 接口来获取到传感器内部的 ADC 数值，从而得到想要测量的结果。Linux 内核为了管理这些日益增多的 ADC 类传感器，特地推出了 IIO 子系统。

IIO 全称是 Industrial I/O，翻译过来就是工业 I/O。一般在搜索 IIO 子系统的时候，会发现大多数讲的都是 ADC，这是因为 IIO 就是为 ADC 类传感器准备的，当然 DAC 也是可以的。大家常用的陀螺仪、加速度计、电压/电流测量芯片、光照传感器、压力传感器等内部都是有个 ADC，内部 ADC 将原始的模拟数据转换为数字量，然后通过其他的通信接口，比如 IIC、SPI 等传输给 SOC。

三、IIO子系统所面向的设备包含：

1. ADC芯片；
2. DAC芯片；
3. 温度传感器；
4. 光感器；
5. 陀螺仪；
6. 加速度计；
7. CDCs；
8. IMUs
9. 压力传感器
10. ...

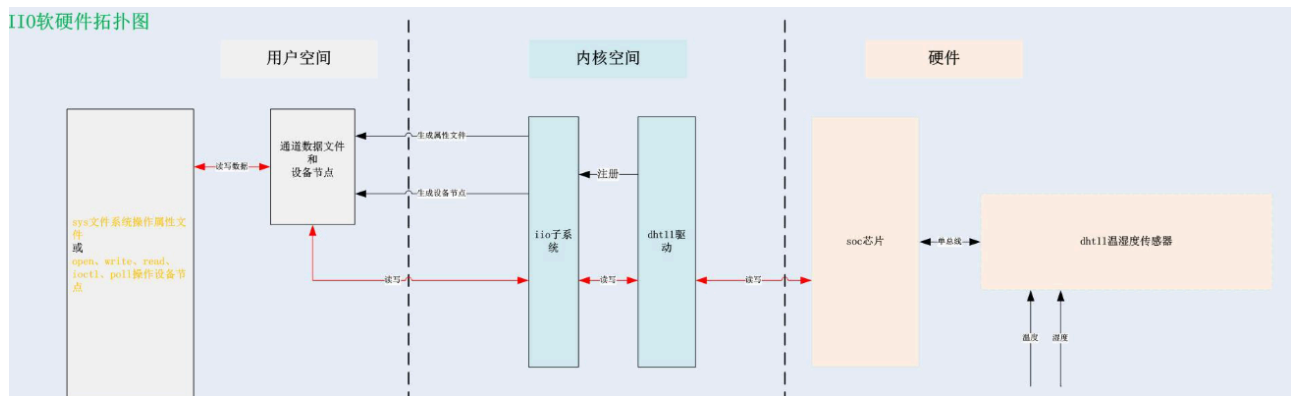
四、IIO子系统与应用层交互的方式：

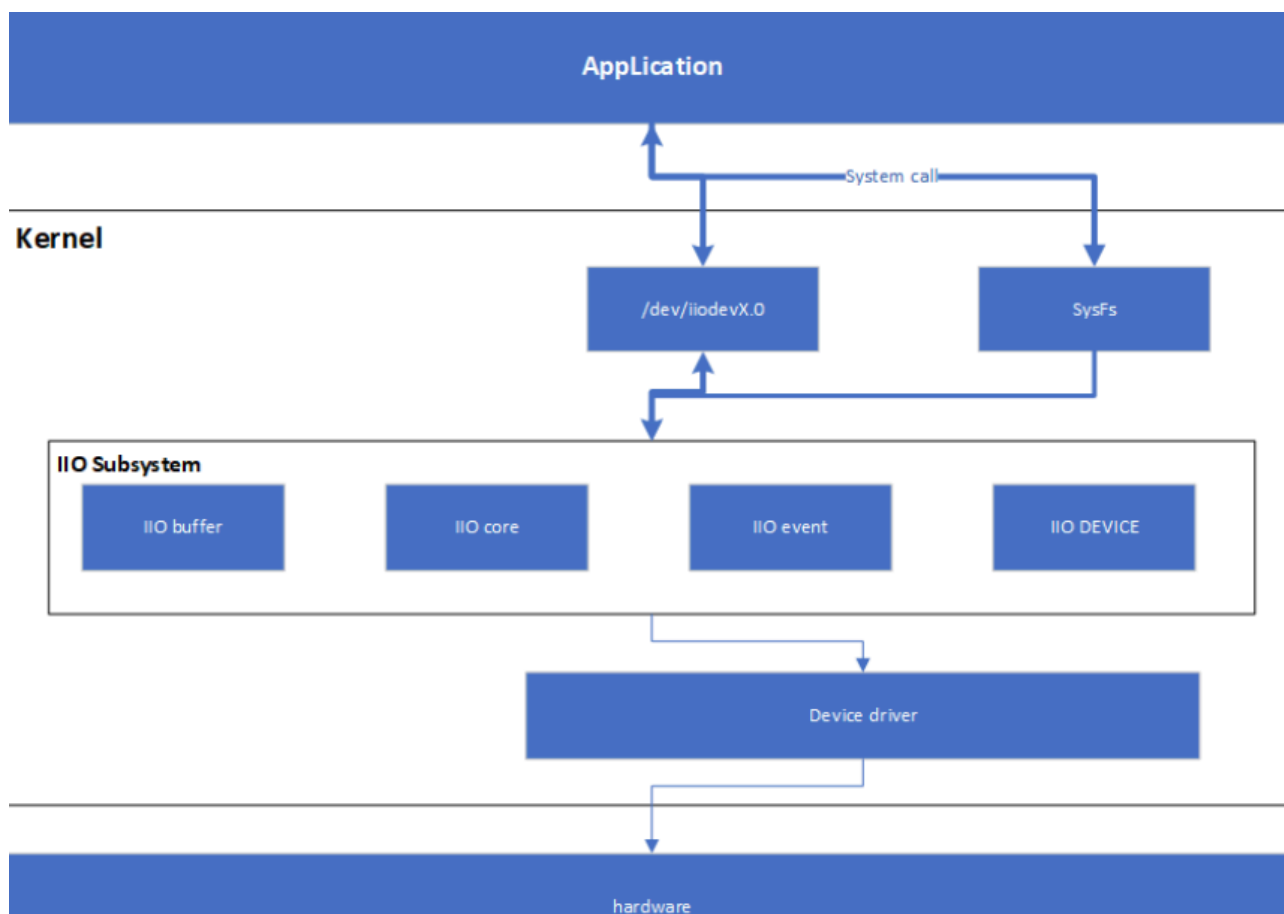
IIO子系统框架中，提供两种方式与应用层交互。

(1) 通过字符设备文件，应用程序可读取IIO device 各通道的信息；`/dev/iio:device`；通过`open`、`read`、`ioctl`函数操作`/dev/iio:device0`等设备节点号获取传感器数据

(2) 通过`sysfs`下的属性文件，应用程序也可以读取IIO device各通道的信息；通过`cat /sys/bus/iio/devices/iio:device`属性文件获取对应传感器数据

五、系统拓扑结构





- (1) 通过将传感器抽象为iio设备后，最终会将其注册为一个字符设备或属性文件；
- (2) 芯片和传感器之间的接口可以是多种形式，如i2c、spi、adc。
- (3) 通过iio可以将传感器的物理数据建立一个数据模型，iio将其定义通道，如dht11可以采集温度和湿度则定义了温度和湿度通道，其它传感器同理；

六、IIO 子系统组成：

1. **IIO buffer:** iio buffer用于处理需要进行连续采集的数据，当一个IIO device的各通道数据支持连续采集时，则调用iio buffer模块提供的接口，创建iio buffer用于存储连续存储的数据，同时该模块提供字符设备文件的注册，因此应用程序通过字符设备文件即可读取该IIO device各通道连续采集的数据。
2. **IIO DEVICE:** 当IIO DEVICE支持连续采集时，一般需要绑定一种触发方法，作为数据到达的信号，而IIO子系统提个iio trigger实现该功能，当iio trigger信号到后，则将数据push到iio buffer中(目前IIO子系统提供了iio trigger的注册、注销与查找接口，且系统中已提供了iio-trig-sysfs、iio-trig-gpio、iio-trig-irq等通用的触发方法。

3. **IIO event:** 针对温度传感器等器件，也会进行事件触发（如温度传感器设置温度上限阈值，当温度高于该阈值后，则触发中断信号），因此IIO子系统提供了iio event模块，iio event模块提供了字符设备操作接口用于对event的监控（该字符设备操作接口并没有创建字符设备文件，而是在iio buffer的ioctl中开启event monitor，然后调用anon_inode_getfd创建了一个匿名inode节点，并设置inode的ops接口等）。
4. **IIO core:** 则主要实现iio device的创建，并根据传递的参数决定是否需要创建iio buffer等，提供了iio device的内存申请、注册与注销接口。

七、IIO相关数据结构

1. iio_dev 结构体:

IO 子系统使用结构体 iio_dev 来描述一个具体 IIO 设备（linux源码/include/linux/iio/iio.h）

```

struct iio_dev {
    int id;
    struct module *driver_module;

    int modes;
    int currentmode;
    struct device dev;

    struct iio_buffer *buffer;
    int scan_bytes;
    struct mutex mlock;

    const unsigned long *available_scan_masks;
    unsigned masklength;
    const unsigned long *active_scan_mask;
    bool scan_timestamp;
    unsigned scan_index_timestamp;
    struct iio_trigger *trig;
    bool trig_readonly;
    struct iio_poll_func *pollfunc;
    struct iio_poll_func *pollfunc_event;

    struct iio_chan_spec const *channels;
    int num_channels;

    const char *name;
    const char *label;
    const struct iio_info *info;
    clockid_t clock_id;
    struct mutex info_exist_lock;
    const struct iio_buffer_setup_ops *setup_ops;
    struct cdev chrdev;
#define IIO_MAX_GROUPS 6
    const struct attribute_group *groups[IIO_MAX_GROUPS + 1];
    int groupcounter;

    unsigned long flags;
    void *priv;
};

```

id: 设备ID号

modes: 设备支持的模式

模式	描述
INDIO_DIRECT_MODE	提供 sysfs 接口
INDIO_BUFFER_TRIGGERED	支持硬件缓冲触发
INDIO_BUFFER_SOFTWARE	支持软件缓冲触发
INDIO_BUFFER_HARDWARE	支持硬件缓冲区

currentmode: 设备当前模式

buffer: 缓冲区

scan_bytes : 捕获到，并且提供给缓冲区的字节数。

available_scan_masks : 可选的扫描位掩码，使用触发缓冲区的时候可以通过设置掩码来确定使能哪些通道，使能以后的通道会将捕获到的数据发送到 IIO 缓冲区。

active_scan_mask : 缓冲区已经开启的通道掩码。只有这些使能了的通道数据才能被发送到缓冲区。

scan_timestamp : 扫描时间戳，如果使能以后会将捕获时间戳放到缓冲区里面。

trig : IIO 设备当前触发器，当使用缓冲模式的时候。

pollfunc : 一个函数，在接收到的触发器上运行。

channels : IIO 设备通道，为 iio_chan_spec 结构体类型，稍后会详细讲解 IIO通道。

num_channels : IIO 设备的通道数。

name : IIO 设备名字。

info : iio_info 结构体类型，这个结构体里面有很多函数，需要驱动开发人员编写，非常重要！我们从用户空间读取 IIO 设备内部数据，最终调用的就是 iio_info 里面的函数。稍后会详细讲解 iio_info 结构体。

setup_ops : iio_buffer_setup_ops 结构体类型，内容如下：

```

struct iio_buffer_setup_ops {
    int (*preenable)(struct iio_dev *);
    int (*postenable)(struct iio_dev *);
    int (*predisable)(struct iio_dev *);
    int (*postdisable)(struct iio_dev *);
    bool (*validate_scan_mask)(struct iio_dev *indio_dev,
                               const unsigned long *scan_mask);
};

```

`iio_buffer_setup_ops` 里面都是一些回调函数，在使能或禁用缓冲区的时候会调用这些函数。如果未指定的话就默认使用 `iio_triggered_buffer_setup_ops`。

2. `iio_dev` 申请与释放：

在使用之前要先申请 `iio_dev` 结构体，申请函数为 `iio_device_alloc` 函数：

```

struct iio_dev *iio_device_alloc(struct device *parent, int sizeof_priv);

```

功能：

为 `iio_dev` 结构体及其私有数据分配内存，对 `iio_dev` 结构体进行初始化，为设备驱动私有数据预留空间

参数：

parent: 指定新分配的IIO设备的父设备

sizeof_priv: 私有数据内存空间大小，一般我们会将自己定义的设备结构体变量作为 `iio_dev` 的私有数据，这样可以直接通过 `iio_device_alloc` 函数同时完成 `iio_dev` 和设备结构体变量的内存申请。申请成功以后使用 `iio_priv` 函数来得到自定义的设备结构体变量首地址。

返回值：

成功返回 `iio_dev` 首地址，失败返回 `NULL`

BH1750(IIO 子系统实现)

一、向rk3588设备树文件中的i2c6控制器节点下增加bh1750传感器节点：

二、编译内核

顶层目录：./build kernel，output/fireware中会生成内核映像文件boot.img (包含内核+设备树文件)

三、通过RKDevTool工具烧录编译号的boot.img镜像(Windows)



注：需要先安装一个RockChip USB驱动，按住RK3588 V+键开发板上电，进入Loader模式下载boot.img（配置好路径，勾选好要烧写的镜像，点击空白处导出配置.cfg文件，点击执行）

四、重启开发板

1. 查看/proc/device-tree/i2c@fec80000目录下出现bh1750节点，可以使用cat 查看节点中对应属性值 (因为我设备树节点中是将bh1750加到i2c6控制器下，对应的address是fec80000，视情况而定)


```
root@ATK-DLRK3588:/proc/device-tree# ls i2c\@f
i2c@fd880000/ i2c@feaa0000/ i2c@feac0000/ i2c@fec80000/ i2c@fec90000/
i2c@fea90000/ i2c@feab0000/ i2c@fead0000/ i2c@fec80000/
root@ATK-DLRK3588:/proc/device-tree# ls i2c\@fec80000/
'#address-cells'    fusb302@22          name                reg
clock-names         gt911@14            phandle             rtc@51
clocks              interrupts          pinctrl-0           '#size-cells'
compatible          light-sensor@23     pinctrl-names       status
root@ATK-DLRK3588:/proc/device-tree#
```

五、基于IIO子系统实现bh1750驱动采集

1. RK3588原理图，I2C6对应的引脚如图：